CoProof — Plan de Calidad basado en el Joel Test

Equipo: David López 22310432, Daniel Tejeda 22310431, Emiliano Flores 22110044
Fecha de evaluación: Abril 2026

Resumen ejecutivo

El Joel Test define 12 prácticas cuya presencia o ausencia es un indicador directo de la madurez del proceso de desarrollo. Este documento evalúa el estado actual del proyecto CoProof frente a cada punto, justifica cada veredicto con evidencia del repositorio, y propone un plan de acción para los puntos no cubiertos. Se concluye con una tabla resumen.

Evaluación detallada

1. ¿Usas control de versiones?

Estado: ✅ **Cumplido**

El proyecto no solo usa Git como sistema de control de versiones local, sino que GitHub es un componente de arquitectura de primer nivel. El servidor backend implementa autenticación OAuth contra GitHub, crea repositorios por proyecto, administra ramas y pull requests mediante la API REST de GitHub, y procesa webhooks `push` para reindexar el contenido Lean. Esto queda evidenciado en `server/app/api/`, los servicios `git_service` y `validation_service`, y la integración descrita en `docs/arquitectura_proyecto.md`. La pérdida total de código es prácticamente imposible en este contexto.

---

2. ¿Puedes hacer un build en un solo paso?

Estado: ✅ **Cumplido**

El archivo `docker-compose.yml` en la raíz del repositorio orquesta todos los servicios del sistema: `web` (Flask), `celery_worker`, `lean-worker`, `computation-worker`, `frontend` (Angular + Nginx), `db` (PostgreSQL) y `redis`. Un único comando:

```
docker compose up --build
```

compila todas las imágenes, aplica migraciones y levanta el stack completo listo para uso. No requiere pasos manuales adicionales. Cumple el criterio de build reproducible en un paso incluso en una máquina limpia.

---

3. ¿Haces builds diarios?

Estado: ✅ **Implementado**

El archivo `.github/workflows/ci.yml` está activo y se dispara en cada `push` a `main` o `develop` y en cada Pull Request. El pipeline se divide en dos jobs paralelos:

Job `build` (backend):

1. `actions/checkout` — clona el repositorio.
2. `docker compose build` — compila todas las imágenes del stack.
3. `docker compose run --rm web python -m pytest tests/ -v` — ejecuta los tests de integración del API Flask contra una base de datos PostgreSQL real (levantada automáticamente como dependencia del servicio `web`).
4. `docker compose down -v` — limpia los volúmenes después de los tests.

Job `frontend` (Angular + Vitest):

1. `actions/setup-node@v4` con Node 22 y caché de `npm`.
2. `npm ci` — instalación determinista de dependencias.
3. `npx ng test --configuration=ci` — ejecuta los tests unitarios con la configuración CI de Angular (Vitest, single-run, sin watch).
4. `npm run build -- --configuration=production` — verifica que el bundle de producción compila sin errores TypeScript.

Cualquier fallo en cualquier job bloquea el merge. Los errores se detectan en menos de 5 minutos desde el push.

---

4. ¿Tienes una base de datos de bugs?

Estado: ✅ Implementado

El sistema completo de seguimiento de bugs está activo en GitHub Issues:

- Plantilla `.github/ISSUE_TEMPLATE/bug_report.md` activa para todos los reportes nuevos.
- Etiquetas de servicio y prioridad configuradas (`bug`, `backend`, `frontend`, `lean-worker`, `priority:high`, `priority:low`, `regression`, `needs-repro`, `ux`).
- Milestone `Zero Known Bugs` creado. Cada Issue con label `bug` se asigna a este Milestone. Antes de cada entrega el Milestone debe estar en 0 issues abiertos.
- Branch protection en `main` activa: requiere CI verde y 1 review aprobada antes de cualquier merge.

Uso cotidiano: Cada vez que se encuentre un bug, abrir un Issue antes de tocar código. Asignarlo al Milestone `Zero Known Bugs` y a quien lo resolverá. Cerrar el Issue desde el commit con `Closes #N`.

Referencia de etiquetas:

Etiqueta	Color	Significado
`bug`	`#d73a4a`	Error confirmado en el sistema
`regression`	`#e4e669`	Funcionalidad que antes funcionaba
`lean-worker`	`#0075ca`	Bug en el servicio Lean
`backend`	`#5319e7`	Bug en Flask/API

Etiqueta	Color	Significado
`frontend`	`#f9d0c4`	Bug en Angular
`priority:high`	`#b60205`	Bloquea entrega o demo
`priority:low`	`#c2e0c6`	No urgente
`needs-repro`	`#e99695`	No se ha podido reproducir
`ux`	`#bfd4f2`	Problema de usabilidad detectado en sesión

---

5. ¿Corriges bugs antes de escribir código nuevo?

Estado: ✅ **Implementado**

La regla de prioridades está documentada en el `README.md` raíz del repositorio y es visible para todo el equipo en la página principal del repo en GitHub:

> **Regla de prioridades:** Antes de iniciar cualquier historia de usuario nueva, el tablero de Issues no debe tener bugs etiquetados con `priority:high` en estado abierto. Si los hay, el miembro asignado los resuelve primero.

El sistema de Issues con etiquetas de prioridad (creado en el punto 4) es la herramienta que hace esta regla operativa.

---

6. ¿Tienes un calendario actualizado?

Estado: ✅ **Implementado**

El archivo `ROADMAP.md` en la raíz del repositorio define el calendario completo hasta la entrega final del 7 de mayo de 2026:

- **Sprint 1** (14–20 Abr): vistas, acceso, perfil, visuals.
- **Sprint 2** (21–27 Abr): lenguaje natural, traductor formal, LaTeX viewer.
- **Sprint 3** (28–30 Abr): sugerencias IA, modo MPI/Slurm.
- **Feature freeze**: 30 de Abril.
- **Semana de testing**: 30 Abr — 6 May.
- **Entrega final**: 7 de Mayo, tag `v1.0.0`.

Cada sprint tiene un criterio de cierre medible: Milestone del sprint en 0 issues `priority:high` abiertos. El ROADMAP se actualiza al inicio de cada sprint si el scope cambia.

---

7. ¿Tienes una especificación?

Estado: ✅ **Cumplido**

El proyecto cuenta con documentación de especificación en múltiples niveles:

- **Historias de usuario:** `docs/historias-usuario.tex` cubre flujos completos con criterios de aceptación medibles por cada historia.

- **Arquitectura:** `docs/arquitectura_proyecto.md` define componentes, responsabilidades, patrones de diseño y contratos de capa.
- **Diseño de backend:** `docs/diseño_backend_avance_2.md` especifica el esquema de base de datos, los triggers, las entidades y las decisiones técnicas con justificación.
- **Wireframes:** El directorio `wireframes/` contiene prototipos HTML de todas las vistas principales del sistema.
- **Diagramas:** `docs/diagrama_clases_server.tex`, `diagrama_clases_frontend.tex`, `diagrama_clases_lean.tex` describen la estructura orientada a objetos de cada servicio.

La existencia de esta documentación previa al código evita los problemas de diseño tardío que Spolsky señala en este punto. El riesgo es que la documentación quede desactualizada; se recomienda actualizar el documento de arquitectura cada vez que se modifique un contrato de servicio.

---

8. ¿Los programadores tienen condiciones de trabajo tranquilas?

Estado: 🚫 No aplica en su totalidad — Explicación

Este punto asume un entorno laboral donde la gerencia puede decidir sobre oficinas, cubículos o espacios abiertos. En un proyecto escolar, las condiciones de trabajo son responsabilidad individual de cada integrante del equipo y no pueden ser controladas a nivel de proyecto.

Sin embargo, el principio subyacente —proteger el estado de *flow* de los programadores— sí puede aplicarse de forma adaptada:

- **Acordar bloques de trabajo sin interrupciones:** Definir ventanas de trabajo concentrado (por ejemplo, sábados de 10:00 a 13:00) en las que el equipo no se interrumpe entre sí por mensajes de texto.
- **Comunicación asíncrona por defecto:** Preferir Issues y comentarios en PRs sobre llamadas de voz no agendadas para preguntas que no son urgentes.

Este punto no puede implementarse completamente por la naturaleza del proyecto, pero el principio puede adoptarse parcialmente.

---

9. ¿Usas las mejores herramientas que el dinero puede comprar?

Estado: ✅ Cumplido

El stack tecnológico es moderno, bien soportado y adecuado para el problema:

- **Lean 4** para verificación formal: es el estado del arte en asistentes de prueba interactivos.
- **Flask + SQLAlchemy + Alembic** para el backend: maduros, con ecosistema amplio y migraciones versionadas.
- **Angular 19** con standalone components para el frontend: framework empresarial con tipado fuerte.
- **PostgreSQL** como base de datos relacional con soporte JSONB y triggers nativos.
- **Redis + Celery** para colas de tareas asíncronas: solución probada a escala.
- **Docker + Docker Compose** para desarrollo reproducible y despliegue.
- **GitHub** como repositorio remoto y plataforma de colaboración.

Todas estas herramientas son de acceso libre o gratuito para equipos pequeños y de código abierto, eliminando la barrera económica. El equipo no usa herramientas desactualizadas ni workarounds por falta de acceso a mejores alternativas.

---

10. ¿Tienes testers?

Estado: ✅ Implementado

Con 3 integrantes no hay un tester dedicado, pero el proceso equivalente está completamente formalizado y activo:

- **Branch protection en `main`** exige 1 review aprobada de un compañero distinto al autor antes de cualquier merge. El revisor ejecuta `docker compose up --build` y prueba manualmente el flujo afectado.
- **Plantilla de PR** (`.github/PR_TEMPLATE/pull_request_template.md`) con checklist obligatoria: build no roto, flujo probado manualmente, sin secretos en código, endpoints con validación, Issue referenciado con `Closes #N`.
- **CI automático** bloquea cualquier PR con tests fallidos, actuando como primera línea de testing antes de que llegue a revisión humana.
- **Rotación de rol tester:** durante la semana de testing (30 Abr–6 May), cada miembro dedica al menos una sesión completa a probar flujos de usuario en el frontend y registrar defectos como Issues.

---

11. ¿Los candidatos escriben código durante la entrevista?

Estado: 🚫 No aplica — Explicación

Este punto es irrelevante por la naturaleza del proyecto. El equipo no tiene un proceso de contratación porque fue asignado o formado al inicio del semestre. No existe un proceso de selección de candidatos que diseñar ni implementar. Ninguna adaptación de este punto tiene sentido en el contexto de un proyecto escolar con equipo fijo.

---

12. ¿Haces pruebas de usabilidad en el pasillo?

Estado: ✅ Implementado — Sesiones agendadas

Se han definido dos sesiones de prueba de usabilidad con personas externas al equipo de desarrollo, alineadas con el calendario del proyecto:

Sesión 1 — Pre-demo (semana del 28 Abr)

- **Participantes:** 2–3 compañeros de clase del mismo semestre.
- **Tarea a observar:** "Entra a la plataforma con tu cuenta de GitHub, crea un proyecto nuevo, agrega un nodo hoja con una proposición simple y valida la prueba."
- **Observador:** un miembro del equipo (sin intervenir, solo registrar dónde el usuario se detiene o duda).
- **Resultado esperado:** Issues abiertos con label `ux` para cada punto de fricción detectado.

Sesión 2 — Semana de testing (30 Abr–6 May)

- **Participantes:** 2–3 personas distintas a la sesión 1 (preferentemente alguien ajeno a programación o matemáticas formales).
- **Tarea a observar:** flujo completo de colaboración — invitar a un colaborador, editar un nodo existente y hacer merge de la propuesta.
- **Resultado esperado:** lista final de problemas de UX resuelta antes del 6 de Mayo.

Protocolo de cada sesión:

1. El participante recibe solo la URL de la plataforma y la descripción de la tarea. Sin tutoriales previos.
2. El observador no interviene ni da pistas durante la sesión.
3. Al finalizar, el observador abre un Issue por cada punto de fricción con label `ux` y `priority:high` o `priority:low` según impacto.
4. Los Issues `ux` se resuelven antes de la entrega final.

Con 5 participantes en total distribuidos entre ambas sesiones, se cubre el umbral de Jakob Nielsen (85% de problemas de usabilidad detectables).

Tabla resumen

#	Punto	Estado	Justificación
1	Control de versiones	✅ Cumplido	Git + GitHub como componente de arquitectura central; integración OAuth, PRs y webhooks.
2	Build en un paso	✅ Cumplido	`docker compose up --build` construye y levanta todo el stack desde cero.
3	Builds diarios / CI	✅ Implementado	`.github/workflows/ci.yml` activo: build Docker, tests backend (pytest + PostgreSQL real), tests frontend (Angular/Vitest CI config), build de producción.
4	Base de datos de bugs	✅ Implementado	Issues con plantilla, etiquetas completas, Milestone `Zero Known Bugs` y branch protection activos.
5	Bugs antes de código nuevo	✅ Implementado	Regla documentada en README.md raíz, visible en la página del repositorio.
6	Calendario actualizado	✅ Implementado	ROADMAP.md con sprints, feature freeze (30 Abr) y entrega final (7 May).
7	Especificación / Spec	✅ Cumplido	User stories, arquitectura, diseño de BD, wireframes y diagramas de clases documentados.
8	Condiciones tranquilas	🚫 No aplica completamente	Proyecto escolar; condiciones individuales. Se puede adoptar el principio con bloques de trabajo acordados.
9	Mejores herramientas	✅ Cumplido	Stack moderno: Lean 4, Angular, Flask, PostgreSQL, Redis, Docker, GitHub — todas gratuitas.
10	Testers	✅ Implementado	Branch protection con review obligatoria, PR checklist, CI automático y rotación de rol tester en semana de testing.

#	Punto	Estado	Justificación
11	Código en entrevistas	🚫 No aplica	Equipo escolar fijo, sin proceso de contratación posible ni necesario.
12	Usabilidad en el pasillo	✅ Implementado	Dos sesiones agendadas: semana del 28 Abr y semana de testing (30 Abr–6 May), ~5 participantes externos en total.

Puntos implementados: 10 / 12 (1, 2, 3, 4, 5, 6, 7, 9, 10, 12)
Puntos que no aplican por naturaleza del proyecto: 2 / 12 (8 y 11)
Total: 10 / 12 — objetivo cumplido

Priorización del plan

Todos los puntos implementables están completados. Las únicas acciones pendientes son operativas:

1. ~~**CI (punto 3)**~~ ✅ Completado.
2. ~~**Bug database (punto 4)**~~ ✅ Completado — Issues, etiquetas, Milestone y branch protection activos.
3. ~~**Bugs antes de código nuevo (punto 5)**~~ ✅ Completado — README actualizado.
4. ~~**Calendario (punto 6)**~~ ✅ Completado — ROADMAP.md creado.
5. ~~**Testers / revisión cruzada (punto 10)**~~ ✅ Completado — branch protection con review obligatoria activa.
6. ~~**Usabilidad en el pasillo (punto 12)**~~ ✅ Completado — dos sesiones agendadas.

Acción continua: registrar bugs como Issues en GitHub a medida que se encuentren durante el desarrollo y la semana de testing.